

Production-Grade RAG

From Scratch -> Evaluated -> Deployed -> Published

Project Playbook, Experiment Log, Weekly To-Do List and Publication Plan

Reference architecture: Ashok007-cmd/production-grade-rag

Implementation goal: independently re-create the engineering ideas, codebase and deployment with original implementation choices.

Working baseline decisions

Vector DB: Qdrant | Corpus: BEIR FiQA-2018 | Local LLM: Ollama | Hosted provider: Cohere | Backend: FastAPI |
Hosting: Render | CI/CD: GitHub Actions

Version 1.0 | 27 September 2026

Contents

- 1. Project purpose and definition of done
- 2. Finalized design decisions
- 3. Final target architecture
- 4. Repository and code organization
- 5. Corpus decision: FiQA-2018
- 6. Environment and tooling setup
- 7. 8 core build weeks + 2 consolidation weeks
- 8. Experiment backlog and experiment protocol
- 9. Evaluation framework and benchmark plan
- 10. Docker learning and migration plan
- 11. Qdrant Cloud step-by-step runbook
- 12. Render deployment step-by-step runbook
- 13. GitHub Actions CI/CD plan
- 14. Observability and production-readiness checklist
- 15. Publishing the GitHub repository
- 16. Blog series and LinkedIn launch plan
- 17. Extension path: Agentic RAG
- 18. Break-friendly operating model and tracking sheet
- 19. Troubleshooting playbook
- 20. Final definition of done
- 21. Current reference links

How to use this document: Treat each checkbox as a task. Complete the weekly exit criteria before moving forward. Office work may create gaps of days or weeks; the plan is sequence-based rather than calendar-dependent. When a task involves unfamiliar infrastructure, use the included runbook first, then ask for step-by-step guidance before making production changes.

1. Project purpose and definition of done

1.1 Why this project exists

This project is meant to become a portfolio-quality AI Engineering artifact that demonstrates much more than “I built a RAG chatbot.” The target story is that a senior data scientist can take retrieval research concepts, implement the components independently, evaluate them quantitatively, expose them as a service, add production safeguards, deploy the application, and explain the trade-offs.

- **Primary learning goal:** move from RAG concepts to end-to-end AI application engineering.
- **Primary portfolio goal:** publish an original GitHub repository with reproducible experiments and a live demo.
- **Primary research goal:** measure dense retrieval, lexical retrieval, hybrid retrieval and reranking rather than treating them as black boxes.
- **Primary deployment goal:** run a public FastAPI application without making Docker, cloud networking or Kubernetes prerequisites for the first half of the project.
- **Future goal:** extend the same retrieval stack into an Agentic RAG use case instead of throwing the work away and starting a new project.

1.2 Final deliverables

- ☐ Public GitHub repository with clear README, architecture, setup instructions and experiment results.
- ☐ Working RAG pipeline using FiQA-2018 and Qdrant.
- ☐ Dense retrieval baseline with a documented embedding model.
- ☐ BM25 lexical retrieval baseline.
- ☐ Hybrid retrieval with reciprocal rank fusion (RRF).
- ☐ Reranking stage with at least one reranker.
- ☐ Quantitative retrieval evaluation against FiQA relevance judgments.
- ☐ Generation evaluation with a small, reproducible evaluation suite.
- ☐ FastAPI backend with health/readiness endpoints and query endpoint.
- ☐ Streaming response path, preferably Server-Sent Events (SSE).
- ☐ Basic request validation, error handling, logging and secret management.
- ☐ Unit and integration tests.
- ☐ Docker packaging added after the application works natively.
- ☐ GitHub Actions CI workflow.
- ☐ Optional quality-gate workflow for retrieval evaluation.
- ☐ Public cloud deployment of the backend on a free/hobby tier.
- ☐ Persistent vector store in Qdrant Cloud for the hosted demo.
- ☐ Simple browser UI or minimal frontend for demoability.
- ☐ Observability/tracing sufficient to show retrieval, reranking and generation latency.
- ☐ Technical blog series and LinkedIn launch post.
- ☐ Short roadmap for transforming the application into Agentic RAG.

1.3 Definition of done for the portfolio

Dimension	Done means
Code originality	Core retrieval, pipeline, API and evaluation logic were implemented independently; the reference repo was used only as an architecture/feature reference.
Reproducibility	Another developer can clone the repo, configure environment variables and run the application using the README.
Evidence	Every major design choice has either a reason, an experiment, or a documented constraint.
Evaluation	The repo shows retrieval metrics and generation-quality metrics, not only screenshots.
Production mindset	The repo includes validation, tests, health checks, logging,

	secrets management, CI and deployment notes.
Demoability	A reviewer can send a question and see an answer with sources without reading the code first.
Communication	The blog explains architecture, experiments, lessons and trade-offs; the LinkedIn post points to measurable results.

2. Finalized design decisions

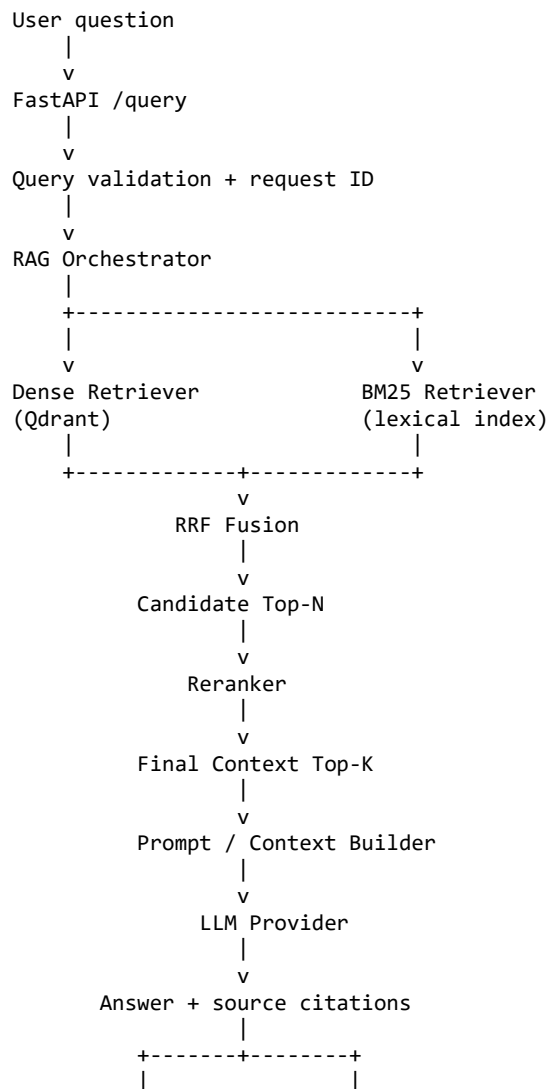
Decision	Final choice	Reason / implication
Vector database	Qdrant	Strong fit for hybrid retrieval, filtering and cloud migration; existing familiarity reduces setup risk. Qdrant also supports local embedded mode, so Docker is not required at the beginning. [1][2]
Alternative vector DB	Do not add Chroma for now	Chroma is a good teaching tool, but using both would add complexity without improving the project outcome. A deliberate Qdrant choice produces a cleaner architecture story.
Local vector storage	Qdrant local mode	Use in-memory mode for disposable experiments and local persistent storage for development state. [2]
Cloud vector storage	Qdrant Cloud Free	Current free cluster: 0.5 vCPU, 1 GB RAM, 4 GB disk, single node; suitable for prototype/testing. Free clusters can suspend after 1 week unused and are deleted after 4 weeks of inactivity if not reactivated. [3][4]
LLM during learning	Ollama	Keeps the baseline cheap and local; lets the code exercise a real provider interface without putting every experiment behind an API key.
Hosted LLM	Cohere first; OpenAI optional	Cohere is especially useful for this project because it also provides reranking. Current evaluation/trial keys are usage-limited, so the public demo should be rate-limited and kept small. [5]
Embedding model	Hugging Face sentence-transformer baseline	Keeps the baseline cost-free and reproducible. Later compare with Cohere embeddings as an experiment rather than making an API provider a hard dependency.
Lexical retrieval	BM25	Transparent baseline and complementary to dense retrieval for exact terms, identifiers and domain-specific vocabulary.
Hybrid fusion	RRF first	Simple, robust and easy to explain. Weighted fusion becomes a later experiment.
Reranking	Cohere Rerank and/or local cross-encoder	Compare quality/latency/cost trade-offs. Cohere trial currently lists 10 rerank requests/minute; plan experiments accordingly. [5]
Backend	FastAPI	Good fit for Python, simple API surface, automatic OpenAPI docs, easy Render deployment. Render explicitly supports FastAPI web services. [6]
Local infrastructure at start	No Docker	Learn RAG first. Docker is introduced after the native application is stable so that it is

		a packaging skill rather than a prerequisite.
Hosting	Render free web service for first public deployment	Simple Git-connected deployment; free web services exist for testing/hobby use and sleep after 15 minutes of inactivity. [6][7]
Static site	GitHub Pages, optional	Useful for project landing page/docs only; GitHub Pages is static hosting, not a Python backend runtime. [8]
CI/CD	GitHub Actions	Run linting/tests/evaluation in CI and later trigger deployment. GitHub Actions supports continuous deployment workflows. [9]

Core architecture principle: Keep the application independent from infrastructure providers. A Qdrant client adapter should not know whether Qdrant is local or cloud; an LLM provider interface should not know whether the implementation is Ollama, Cohere or OpenAI.

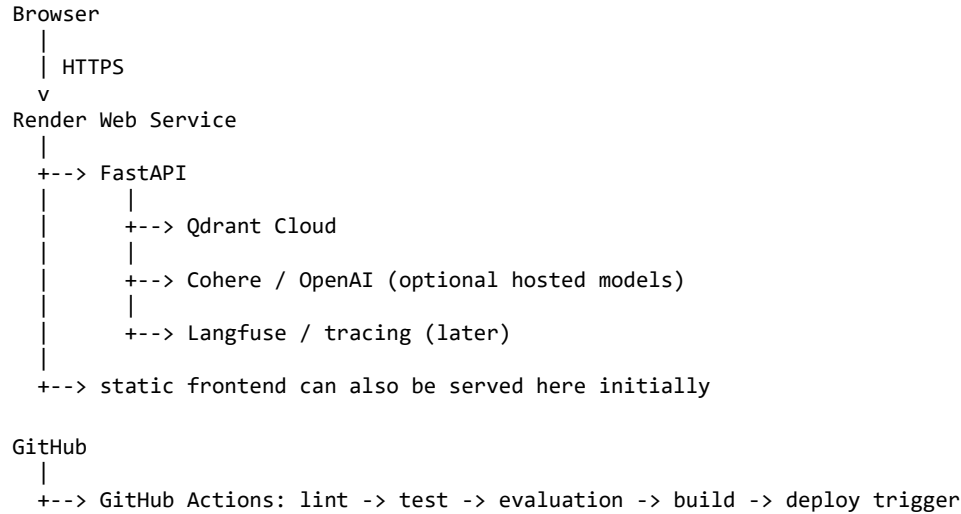
3. Final target architecture

3.1 Logical RAG pipeline

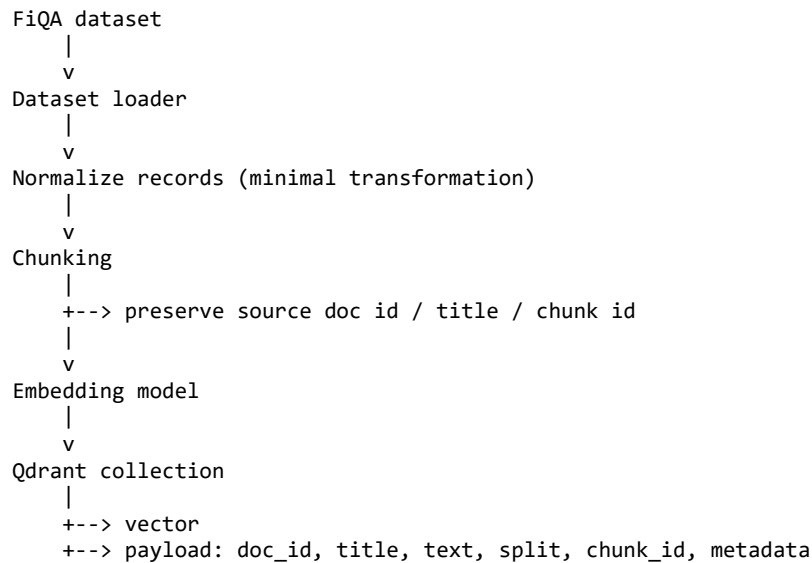


v v
 User answer telemetry/metrics

3.2 Hosted architecture



3.3 Ingestion pipeline



3.4 Important separation of concerns

Component	Responsible for	Must not own
Dataset loader	Reading benchmark records into an internal document model	Vector DB details
Chunker	Splitting text with traceable metadata	Embedding or retrieval logic
Embedder	Turning text into vectors	Choosing where vectors are stored
Qdrant adapter	Create collection, upsert, search, filters	Prompt building
BM25 adapter	Lexical index and lexical retrieval	LLM calls
Fusion	Combining ranked lists	Embedding model
Reranker	Re-scoring candidate passages	Answer generation
Generator	Prompt + LLM response	Retrieval implementation
Evaluator	Metrics and evaluation reports	Serving traffic
API layer	HTTP contract, validation, errors	Experiment implementation

4. Repository and code organization

```

production-rag-fiqa/
|
+-- app/
|   +-- api/
|   |   +-- routes.py
|   |   +-- schemas.py
|   |   +-- errors.py
|   +-- ingestion/
|   |   +-- loaders.py
|   |   +-- chunking.py
|   |   +-- pipeline.py
|   +-- retrieval/
|   |   +-- dense.py
|   |   +-- bm25.py
|   |   +-- fusion.py
|   |   +-- reranking.py
|   +-- generation/
|   |   +-- providers.py
|   |   +-- prompts.py
|   |   +-- citations.py
|   +-- evaluation/
|   |   +-- retrieval_metrics.py
|   |   +-- generation_metrics.py
|   |   +-- runner.py
|   +-- observability/
|   |   +-- logging.py
|   |   +-- tracing.py
|   +-- config/
|   |   +-- settings.py
|
+-- scripts/
|   +-- ingest.py
|   +-- query.py
|   +-- evaluate.py
|   +-- benchmark.py
|
+-- tests/
|   +-- unit/
|   +-- integration/
|   +-- evaluation/
|
+-- experiments/
|   +-- results/
|   +-- notes/
|
+-- data/
|   +-- README.md
|   +-- sample/
|
+-- frontend/
|
+-- .github/workflows/
|   +-- ci.yml
|   +-- eval.yml
|   +-- deploy.yml
|
+-- Dockerfile
+-- docker-compose.yml
+-- .env.example
+-- requirements.txt or pyproject.toml
+-- README.md

```

+-- LICENSE

Data policy: Do not commit large benchmark data, API keys, local vector indexes or generated caches into Git. Commit small evaluation fixtures and scripts; document exactly how the larger FiQA data is downloaded. Before publishing, verify the dataset licensing/attribution terms and comply with the source terms. The current Hugging Face BelR/fiqa dataset card labels it CC BY-SA 4.0, while its discussion points to differences in the original FiQA source terms; record the exact source and license you rely on in the repo. [10][11]

5. Corpus decision: FiQA-2018

5.1 Why FiQA-2018 is the selected corpus

FiQA is the right first benchmark because it combines three things needed for this project: a financial domain, a research benchmark format, and relevance judgments that let us evaluate retrieval objectively. BEIR lists FiQA-2018 with train/dev/test splits, 648 test queries and about 57K corpus documents. The Hugging Face dataset is already structured into corpus and queries; qrels provide relevance judgments. [10][12]

Property	FiQA choice for this project
Domain	Finance / financial information retrieval
Corpus size	57,638 documents in the current Hugging Face dataset card; BEIR summary rounds this to 57K. [10][12]
Test queries	648. [12]
Format	Structured corpus with _id, title and text; queries with _id and text; qrels with query id, corpus id and relevance score. [10][12]
Data preparation	Minimal normalization rather than PDF/OCR/table cleaning.
Research value	Part of BEIR, a heterogeneous IR benchmark; easy to compare retrieval methods in a recognized benchmark setup. [12]
Portfolio value	Finance is relevant to the user's preferred domains and creates a natural bridge to future research/agent use cases.

5.2 Corpus usage plan

- [] Load a tiny sample first (e.g., 500-1,000 documents) to validate the pipeline.
- [] Scale to 5K documents for rapid local experiments.
- [] Scale to 10K-25K for retrieval and latency experiments.
- [] Run final benchmark on the full FiQA corpus if local resources and time allow.
- [] Keep benchmark test queries separate from any development tuning data.
- [] Use qrels for retrieval evaluation and do not tune a model on the test set repeatedly.
- [] Create a small manually inspected set of 20-30 user-style questions for qualitative demo testing.

5.3 Important evaluation discipline

- Use train/dev material for design iteration when feasible; reserve the held-out test queries for final reporting.
- For every experiment record dataset subset, embedding model, chunking config, top-k, fusion weights, reranker and timestamp.
- Do not report one “best” configuration without recording the alternatives tried.
- Separate retrieval quality from generation quality. A good answer does not prove retrieval is good, and a poor answer does not automatically prove retrieval is poor.

6. Environment and tooling setup

6.1 Initial tools to install

- [] Cursor IDE.
- [] Python 3.11 or 3.12.
- [] Git and GitHub account.
- [] A Python virtual environment for the project.
- [] Ollama for local model serving.
- [] A Hugging Face account only if a model/dataset download requires it.
- [] Optional Cohere API key for hosted experiments.
- [] Optional OpenAI API key for a later provider comparison.
- [] Do not install Docker yet unless you want to learn it in parallel; the project is intentionally designed to work without it at first.

6.2 Initial project bootstrap

```
# Windows PowerShell example
mkdir production-rag-fiqa
cd production-rag-fiqa

python -m venv .venv
.venv\Scripts\Activate.ps1

python -m pip install --upgrade pip

# later: install only the dependencies required for the current week
# keep requirements.txt or pyproject.toml under version control
```

Working habit: Install the minimum dependency set for each stage. Avoid adding LangChain, LlamaIndex, multiple vector databases or multiple orchestration frameworks “just in case.” Every dependency should earn its place.

6.3 Provider abstraction from day one

```
class LLMPProvider:
    def generate(self, prompt: str, **kwargs):
        raise NotImplementedError

class EmbeddingProvider:
    def embed_documents(self, texts):
        raise NotImplementedError

    def embed_query(self, text):
        raise NotImplementedError

class RerankerProvider:
    def rerank(self, query, documents, **kwargs):
        raise NotImplementedError
```

The exact class names are not important. The design principle is. This abstraction lets the project move from Ollama/local Hugging Face models to Cohere/OpenAI without rewriting the retrieval pipeline.

7. 8 core build weeks + 2 consolidation weeks

Time-budget rule: Each “week” is a unit of work, not a calendar promise. A week can be completed over several weekends and evenings. When office work interrupts the project, stop at the last checkpoint instead of jumping

ahead.

Week 0 - Architecture, repo and environment

Goal: Create the foundation without implementing the full RAG system.

- ☐ Read the reference repo README and architecture; do not copy implementation code.
- ☐ Create your GitHub repository with a neutral project name.
- ☐ Create a project README with Problem, Goal, Architecture, Roadmap and Status sections.
- ☐ Create .gitignore and .env.example.
- ☐ Create Python virtual environment.
- ☐ Install Cursor Python tooling / formatter / linter of choice.
- ☐ Install and verify Ollama.
- ☐ Install qdrant-client.
- ☐ Create first small Python smoke test.
- ☐ Write PROJECT_SPEC.md capturing reference feature -> your implementation mapping.

Exit criterion: Exit: repository runs a trivial Python script, Git works, Ollama responds, and qdrant-client imports successfully.

Week 1 - Baseline ingestion, chunking and dense retrieval

Goal: Build the first RAG primitive without an LLM.

- ☐ Load a small FiQA sample.
- ☐ Define an internal Document and Chunk data structure.
- ☐ Implement text normalization that preserves IDs and titles.
- ☐ Implement first chunker; start with a simple fixed/recursive approach.
- ☐ Store source document ID and chunk ID on every chunk.
- ☐ Create embedding provider interface.
- ☐ Implement local Hugging Face embedding provider.
- ☐ Create Qdrant collection.
- ☐ Upsert vectors + payload.
- ☐ Implement top-k semantic search.
- ☐ Write a script that prints retrieved chunks for a query.

Exit criterion: Exit: given a natural-language query, the system returns top-k chunks with source IDs and scores.

Week 2 - Generation, citations and FastAPI

Goal: Turn retrieval into a usable RAG response.

- ☐ Implement LLM provider interface.
- ☐ Implement Ollama provider first.
- ☐ Build prompt template with explicit grounding instructions.
- ☐ Pass retrieved contexts to the LLM.
- ☐ Return answer + citations/source IDs.
- ☐ Create FastAPI application.
- ☐ Add /health.
- ☐ Add /ready.
- ☐ Add POST /query.
- ☐ Define request/response schemas.
- ☐ Add basic error handling and structured logs.

[] Verify local API using curl or the FastAPI docs UI.

Exit criterion: Exit: local browser/API call returns an answer with traceable sources and the service has health endpoints.

Week 3 - BM25 and hybrid retrieval

Goal: Add the lexical branch and prove why hybrid retrieval is useful.

- [] Build a BM25 index from the same corpus.
- [] Implement BM25 retriever adapter.
- [] Create a common retrieval result schema.
- [] Implement rank-based reciprocal rank fusion.
- [] Compare dense vs BM25 vs hybrid on selected queries.
- [] Add retrieval metrics to the experiment script.
- [] Record examples where lexical retrieval beats semantic retrieval.
- [] Record examples where semantic retrieval beats lexical retrieval.
- [] Document failure cases rather than only successes.

Exit criterion: Exit: hybrid retrieval works end-to-end and a first quantitative comparison table exists.

Week 4 - Reranking and evaluation harness

Goal: Introduce reranking and build the benchmark loop.

- [] Implement reranker interface.
- [] Add local cross-encoder or Cohere Rerank adapter.
- [] Retrieve a larger candidate set (for example 20-50).
- [] Rerank candidates and pass top-k to the generator.
- [] Load FiQA qrels.
- [] Implement Recall@K, MRR/nDCG as appropriate to the evaluation question.
- [] Create a reproducible evaluation config file.
- [] Run Dense vs BM25 vs Hybrid vs Hybrid+Rerank.
- [] Record latency for retrieval and reranking.
- [] Create a first experiment report under experiments/results/.

Exit criterion: Exit: one command runs an offline retrieval benchmark and produces a results table.

Week 5 - Production API engineering

Goal: Harden the service before deployment.

- [] Separate API, orchestration and provider layers.
- [] Add environment-based configuration.
- [] Remove secrets from source code.
- [] Add request validation and file-size/query-size limits.
- [] Add request IDs.
- [] Add consistent error responses.
- [] Add timeouts to external provider calls.
- [] Add retry strategy only where safe and bounded.
- [] Add streaming endpoint using SSE.
- [] Add API authentication mechanism suitable for a hobby demo if needed.
- [] Add unit tests for chunking, fusion, schema validation and adapters.
- [] Add one integration test for the complete query path.

Exit criterion: Exit: the API survives malformed inputs and provider failures without crashing the process; tests run locally.

Week 6 - Observability, generation evaluation and cost/latency tracking

Goal: Make the system measurable like a real application.

- ☐ Add structured JSON-friendly logging.
- ☐ Record total request latency.
- ☐ Record retrieval latency.
- ☐ Record reranking latency.
- ☐ Record generation latency.
- ☐ Record model/provider used.
- ☐ Record token usage when available.
- ☐ Add Langfuse or another lightweight tracing layer.
- ☐ Create 20-30 manually inspected demo questions.
- ☐ Build or adopt an LLM-as-judge evaluation for faithfulness and answer relevance.
- ☐ Track hallucination/unsupported-answer examples.
- ☐ Define a quality dashboard/report structure.

Exit criterion: Exit: each test query can be traced from API request through retrieval to generation and the repo contains a repeatable quality report.

Week 7 - Docker and GitHub Actions

Goal: Package the working application and automate checks.

- ☐ Learn Docker concepts: image, container, port, volume, environment variable.
- ☐ Install Docker Desktop only now if not already installed.
- ☐ Containerize FastAPI application.
- ☐ Create local docker-compose setup for app + Qdrant where useful.
- ☐ Verify health endpoint inside the container.
- ☐ Make CI workflow run formatting/linting, unit tests and integration tests.
- ☐ Add dependency caching only after the simple workflow works.
- ☐ Add optional evaluation workflow for benchmark quality.
- ☐ Build Docker image in CI.
- ☐ Optionally publish image to GitHub Container Registry.

Exit criterion: Exit: a clean clone can be tested and containerized with documented commands; GitHub Actions is green on every PR/main push.

Week 8 - Qdrant Cloud, Render deployment and demo UI

Goal: Put a safe, reproducible public version online.

- ☐ Create Qdrant Cloud free cluster.
- ☐ Create API key and store it as a deployment secret, not in Git.
- ☐ Make Qdrant connection configurable between local and cloud.
- ☐ Populate the cloud collection with the selected FiQA subset/full benchmark.
- ☐ Create Render Web Service from GitHub.
- ☐ Set environment variables and start command.
- ☐ Configure /health as health check.
- ☐ Verify public HTTPS endpoint.

- ☐ Build minimal frontend.
- ☐ Connect frontend to /query endpoint.
- ☐ Add source citations in UI.
- ☐ Add a “retrieval mode / reranker / latency” debug section if safe for public demo.
- ☐ Test cold start expectations and document them.

Exit criterion: Exit: a reviewer can open the public URL, ask a question, get an answer and inspect sources.

Week 9 - Consolidation, experiments and publication

Goal: Convert the implementation into a strong portfolio story.

- ☐ Clean repository structure and remove dead code.
- ☐ Run final benchmark suite.
- ☐ Freeze the headline experiment configuration.
- ☐ Produce final latency and retrieval tables.
- ☐ Produce a clear architecture diagram.
- ☐ Write final README with quickstart + architecture + results + demo link.
- ☐ Add dataset attribution and license notes.
- ☐ Add known limitations.
- ☐ Draft blog article 1 and publish after technical verification.
- ☐ Draft blog article 2 around retrieval experiments.
- ☐ Draft blog article 3 around productionization.
- ☐ Write LinkedIn launch post with one or two meaningful results.

Exit criterion: Exit: repository is ready for public sharing and the blog can be reproduced from the repo.

8. Experiment backlog and experiment protocol

8.1 Experiment protocol

- ☐ Write one hypothesis sentence before running the experiment.
- ☐ Change one major variable at a time unless explicitly running a factorial experiment.
- ☐ Define the evaluation subset and metric before looking at results.
- ☐ Run the baseline first.
- ☐ Record configuration in a machine-readable file.
- ☐ Record runtime and resource notes, not only quality scores.
- ☐ Save raw result data and a small human-readable summary.
- ☐ Write one paragraph: what changed, why, what happened, what you learned, what you would do next.

Priority	Experiment	Variables	Metrics	Why it matters
P0	Chunk size	256 / 512 / 768 / 1024 tokens; overlap 0 / 50 / 100	Recall@K, nDCG/MRR, answer quality, latency	Teaches a fundamental RAG trade-off and gives a clear blog section.
P0	Retriever comparison	Dense vs BM25	Recall@K, MRR/nDCG, latency	Creates the baseline for hybrid retrieval.
P0	Hybrid fusion	RRF k / weighted alpha	Recall@K, nDCG/MRR, latency	Shows why combining rank signals helps.
P0	Reranker	No reranker vs reranker; candidate top-N	Retrieval metrics, answer quality, latency	Shows quality-latency trade-off.
P1	Embedding models	1-2 HF models vs Cohere embedding	Recall@K, latency, memory/API cost	Demonstrates model selection rather than single-provider dependence.

P1	Top-K sensitivity	K=3/5/8/10 and candidate N=20/50/100	Recall, answer quality, latency	Shows context-size trade-offs.
P1	LLM comparison	Ollama model vs hosted model	Faithfulness, relevance, latency, cost	Links generation behavior to deployment trade-offs.
P1	Prompt variations	Strict grounding vs concise answer vs citation-first	Faithfulness, answer relevance	Shows that generation quality is not only a model problem.
P2	Metadata filtering	No filter vs domain/source filters	Recall, latency	Prepares for multi-tenant or scoped knowledge bases.
P2	Caching	No cache vs semantic/exact cache	Latency, hit rate, cost	Useful production optimization experiment.
P2	Query rewriting	Original query vs rewrite	Recall, answer quality, latency	Bridge to Agentic RAG.
P2	HyDE / multi-query	Single query vs expansion	Recall, quality, latency	Advanced retrieval experimentation; optional.
P3	Document granularity	Passage-level vs larger parent chunks	Recall, context quality	Useful for understanding chunking beyond fixed windows.

8.2 Experiment record template

Experiment ID: EXP-XXX
 Date:
 Hypothesis:
 Dataset split/subset:
 Baseline config:
 Variant config:
 Embedding model:
 Chunking:
 Retriever:
 Fusion:
 Reranker:
 LLM:
 Top-K / Candidate-N:
 Metrics:
 Latency measurements:
 Results:
 Qualitative examples:
 Decision:
 Follow-up:

9. Evaluation framework and benchmark plan

9.1 Retrieval evaluation

Metric	What it answers	Use in project
Recall@K	Did we retrieve at least one relevant document/chunk in top K?	Primary diagnostic for retrieval coverage.
MRR@K	How early did the first relevant result appear?	Useful for understanding ranking quality.
nDCG@K	How good is the ordering when relevance has grades?	Good general-purpose ranking measure when qrels contain relevance scores.
Precision@K	How much of top K is relevant?	Useful secondary metric, especially when context budget matters.
Latency P50/P95	How fast is the system typically / under tail load?	Production trade-off metric.

Use a standard IR library such as `ir-measures` or `pytrec_eval` where useful, but implement at least one simple metric yourself during learning so you understand the calculation and data flow.

9.2 Generation evaluation

Dimension	Question	How to evaluate
Faithfulness / groundedness	Is the answer supported by retrieved context?	LLM-as-judge plus manual spot checks.
Answer relevance	Does the answer address the user question?	LLM-as-judge plus a small human-reviewed set.
Citation correctness	Do cited sources actually support the claim?	Rule-based source linkage + manual inspection.
Abstention behavior	Does the system avoid inventing answers when evidence is insufficient?	Create intentionally unanswerable/weak-evidence questions.

9.3 Quality gates

- [] Do not deploy a change that silently reduces retrieval quality beyond an agreed tolerance.
- [] Do not compare generation quality without using the same retrieval configuration when that is the variable under study.
- [] Track latency with warm and cold conditions separately when deployment platform sleeps/starts on demand.
- [] Keep a small regression suite of questions that previously failed.

10. Docker learning and migration plan

Docker is deliberately delayed. The objective is not to become a Docker expert; it is to understand how to package the application consistently and why containers help deployment.

10.1 Concepts to learn

Term	Plain-English meaning
Image	A packaged filesystem + runtime instructions used to create containers.
Container	A running instance of an image.
Dockerfile	Recipe for building the image.
Port mapping	Connect a port on your laptop/server to the port inside the container.
Volume	Storage mounted outside the disposable container filesystem.
Environment variable	Configuration passed into the container at runtime.
Compose	A simple way to run multiple containers together.

10.2 First Docker session runbook

- [] Install Docker Desktop for Windows when Week 7 starts.
- [] Restart the computer if the installer requests it.
- [] Open Docker Desktop and wait until the engine is running.
- [] Open PowerShell and run `docker --version`.
- [] Run `docker run hello-world`.
- [] Read the output and verify the command exits successfully.
- [] Do not immediately start Docker Compose; first understand image/container/port with one tiny container.
- [] Create a Dockerfile for the FastAPI application.
- [] Build the image locally.
- [] Run the container mapping the FastAPI port to localhost.
- [] Open `/health` in the browser.

[] Only then add Qdrant to a compose file.

```
# Verify Docker
docker --version
docker run hello-world
```

```
# Typical application flow later
docker build -t production-rag:local .
docker run --rm -p 8000:8000 --env-file .env production-rag:local
```

Important: The exact Dockerfile and compose commands will depend on the final app entrypoint and dependency file. Do not blindly copy commands from tutorials. When Week 7 starts, build the Docker setup against the actual repository so each line has a reason.

11. Qdrant Cloud step-by-step runbook

11.1 Goal

Create a managed Qdrant cluster for the hosted demo while keeping local development independent. Qdrant currently documents a free single-node cluster with 0.5 vCPU, 1 GB RAM and 4 GB disk, and no credit card required. [3]

11.2 Account and cluster

- [] Open the official Qdrant Cloud page.
- [] Create/sign into a Qdrant Cloud account.
- [] Choose the Free cluster option.
- [] Select an available cloud provider/region that keeps latency reasonable for the demo.
- [] Create the cluster.
- [] Wait until the cluster is provisioned and marked ready.
- [] Record the cluster URL privately; do not publish secrets.
- [] Generate an API key with the minimum access needed by the application.

11.3 Connect from Python

```
from qdrant_client import QdrantClient
```

```
client = QdrantClient(
    url=settings.qdrant_url,
    api_key=settings.qdrant_api_key,
)
```

- [] Add QDRANT_URL and QDRANT_API_KEY to local .env only for testing.
- [] Add the same values as Render environment variables later.
- [] Run a connectivity smoke test that lists or creates a test collection.
- [] Create the project collection only after connectivity is confirmed.
- [] Delete the smoke-test collection.
- [] Run a small ingestion into the cloud cluster.
- [] Run one query against cloud vectors.
- [] Verify payload metadata and source IDs are correct.

11.4 Capacity rule for the project

- Do not upload unnecessary duplicate vectors.
- Use deterministic collection names and an ingestion version so you can rebuild cleanly.

- For benchmark experiments, consider separate collections for major embedding configurations or wipe/recreate deterministically.
- Because the free cluster is small, start with a subset and only load the full corpus once memory/storage is known to be sufficient.

Current free-tier caveat: Qdrant says unused free clusters are suspended after one week and deleted after four weeks of inactivity if not reactivated. This matters for a portfolio project with long breaks: keep the cluster name/config documented and plan to reactivate/rebuild before a demo. [3]

12. Render deployment step-by-step runbook

12.1 What Render is

Render is a managed hosting platform for web applications and APIs. Its documentation lists Python/FastAPI support, Git-connected deployments and a free web-service tier intended for testing, hobby projects and previews. Free web services spin down after 15 minutes of inactivity, so the first request after a long idle period may be slower. [6][7]

12.2 Prepare the application

- [] Make the application start through a single predictable FastAPI endpoint.
 - [] Add a /health endpoint that does not depend on the LLM generating a response.
 - [] Add /ready if a dependency readiness check is useful.
 - [] Make the server bind to 0.0.0.0.
 - [] Read the port from the PORT environment variable in deployment.
 - [] Move Qdrant and model credentials into environment variables.
 - [] Ensure the app does not try to download a multi-gigabyte local model during Render startup.
 - [] For the first public deployment, prefer hosted LLM/embedding/reranking providers or lightweight remote calls.
- # Example start command pattern
 uvicorn app.main:app --host 0.0.0.0 --port \$PORT

12.3 Create the Render service

- [] Create/sign into Render.
- [] Choose New -> Web Service.
- [] Connect your GitHub account/repository.
- [] Select the project repository and branch.
- [] Set the build command according to the dependency strategy; e.g. pip install -r requirements.txt or pip install .
- [] Set the start command to your FastAPI endpoint.
- [] Choose the Free compute option for the prototype.
- [] Add environment variables for Qdrant, model providers and application configuration.
- [] Set the health check path to /health.
- [] Create the service and watch the first deploy log.

12.4 Validate the deployment

- [] Open the Render public URL.
- [] Check /health.
- [] Open the FastAPI /docs page if exposed.
- [] Run one low-cost query.
- [] Verify citations.
- [] Check logs for secrets accidentally printed.
- [] Stop using any local-only configuration in the production deployment.

[] Wait for idle spin-down and later re-test to understand cold-start behavior.

Hosting principle: Render hosts the application; Qdrant Cloud stores vectors; an external model provider performs hosted inference when selected. Do not make the application depend on the Render local filesystem for durable vector data. GitHub Pages remains an optional static site for the project landing page/docs, not the Python backend. [7][8]

13. GitHub Actions CI/CD plan

13.1 Pipeline stages

```

Pull Request / Push
  |
  v
Install dependencies
  |
  v
Lint / format check
  |
  v
Unit tests
  |
  v
Integration tests
  |
  v
(Optional) retrieval evaluation
  |
  v
Build artifact / Docker image
  |
  v
Deploy trigger

```

13.2 CI checklist

- [] Create .github/workflows/ci.yml.
- [] Trigger on pull_request and push to main.
- [] Set the Python version explicitly.
- [] Install dependencies from the repository file.
- [] Run formatter/linter checks.
- [] Run unit tests.
- [] Run integration tests that do not require paid secrets.
- [] Fail the workflow on test/lint failure.
- [] Keep the workflow output readable and diagnostic.

13.3 Evaluation workflow

- [] Create a separate evaluate.yml workflow after the baseline evaluation command is stable.
- [] Use a small deterministic subset for every commit if cost/time allows.
- [] Use a larger benchmark for manual or scheduled runs.
- [] Store result artifacts so you can compare changes.
- [] Set a tolerance-based quality gate rather than demanding that every change improve every metric.

13.4 Deployment workflow

- [] Start with Render auto-deploys from main rather than building a custom deployment system.

- [] Use GitHub Actions to test before deployment; Render can then deploy the green commit.
- [] If a direct deployment trigger is later needed, use provider-supported actions/webhooks and keep secrets in GitHub/Render secret storage.
- [] Do not store Qdrant, Cohere or OpenAI secrets in repository files.

14. Observability and production-readiness checklist

Area	Minimum project implementation	Later enhancement
Logging	Structured request/error logs, request ID	Centralized log search
Tracing	Trace request -> retrieval -> reranking -> generation	Distributed tracing across services
Metrics	Latency, error rate, token/cost where possible	Prometheus/Grafana style dashboards
Quality	Retrieval and generation evaluation	Automated production feedback loop
Security	Secrets in env/secret store, input validation, safe logging	Rate limiting, stronger auth, WAF
Reliability	Timeouts and bounded retries	Circuit breakers / fallback providers
Caching	Optional exact-query cache	Semantic cache
Deployment	Health check + reproducible start command	Blue/green or staged deployment

- [] No API keys in Git history.
- [] No raw user prompts containing secrets in public logs.
- [] No stack traces returned to public users.
- [] Every request has a way to correlate API response with logs.
- [] Health endpoint is cheap and deterministic.
- [] External API calls have bounded timeouts.
- [] Public demo has a reasonable request/rate limit if the provider key is yours.

15. Publishing the GitHub repository

15.1 Before the repository is public

- [] Search the entire repository for API key patterns, .env files and credential-like strings.
- [] Remove local Qdrant storage from Git history if accidentally committed.
- [] Remove model cache directories and generated output that is not meant for publication.
- [] Confirm dataset attribution and license notes.
- [] Check that README setup commands work from a clean environment.
- [] Verify that the deployment URL in README works.
- [] Add screenshots or a short demo GIF/video if allowed and useful.
- [] Add architecture diagram.
- [] Add experiment results table.
- [] Add “What I learned” and “Limitations” sections.
- [] Add a roadmap pointing to Agentic RAG.

15.2 README structure

```
# Production-Grade RAG on FiQA

## Demo
## Problem
## What this project demonstrates
## Architecture
```

```

## Retrieval pipeline
## Evaluation
## Experiments
## Results
## Tech stack
## Local setup
## Docker setup
## Cloud deployment
## Configuration
## API examples
## Limitations
## Future: Agentic RAG
## Dataset and attribution
## Blog series

```

16. Blog series and LinkedIn launch plan

16.1 Blog article 1 - Build the RAG engine

Suggested title: “I Built a Financial RAG System From Scratch: What Happens Between a Question and an Answer?”

- [] Explain the problem and why FiQA was chosen.
- [] Explain document -> chunk -> embedding -> vector search.
- [] Show one query and retrieved chunks.
- [] Explain Qdrant and why it was chosen over Chroma for this project.
- [] Show the first baseline architecture.
- [] Include one or two early failure cases.

16.2 Blog article 2 - Retrieval experiments

Suggested title: “Dense vs BM25 vs Hybrid Retrieval: What My FiQA Experiments Showed”

- [] Explain dense retrieval and lexical retrieval simply.
- [] Introduce the benchmark and qrels.
- [] Show Dense vs BM25 results.
- [] Show RRF results.
- [] Show reranking impact.
- [] Show latency trade-offs.
- [] Include qualitative query examples where methods disagree.
- [] Do not hide negative results.

16.3 Blog article 3 - Productionization

Suggested title: “From RAG Prototype to Production: FastAPI, Evaluation, Observability, Docker and CI/CD”

- [] Explain why the notebook prototype was insufficient.
- [] Show architecture evolution.
- [] Explain FastAPI and health/readiness endpoints.
- [] Explain Docker at a beginner-friendly level.
- [] Explain Qdrant Cloud and Render roles.
- [] Show CI flow.
- [] Show observability/tracing.
- [] Discuss free-tier limitations and cold starts.

16.4 LinkedIn launch post

The LinkedIn post should be a compact project announcement, not a full tutorial. Use one architecture image, one headline metric/result, the live demo link and the GitHub link. Frame the key takeaway around an engineering lesson, e.g. retrieval quality

improved materially when lexical and semantic signals were combined, or a reranker improved ranking at a measurable latency cost. Use only your actual measured results.

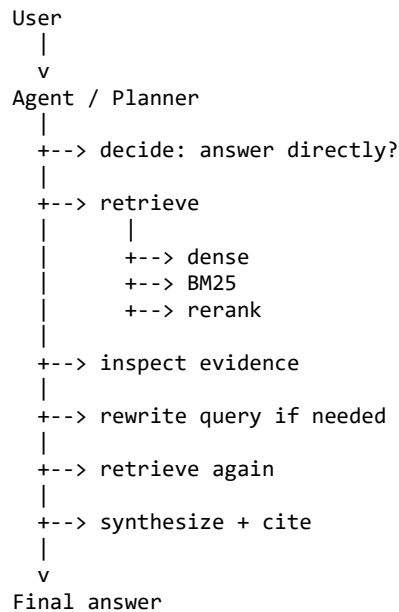
17. Extension path: Agentic RAG

The project should be designed so Agentic RAG becomes an extension of the current system. Do not build agents until the retrieval baseline is measurable and stable.

17.1 Reusable components

Current component	Future agentic use
Retriever interface	Agent can call retrieval as a tool.
Query rewriter	Agent can decide when a question needs query reformulation.
Reranker	Agent can invoke high-precision retrieval before final synthesis.
Evaluator	Agentic workflows can be compared against the same benchmark.
Source/citation layer	Agent can cite evidence from multiple retrieval steps.
Observability	Trace tool calls and agent decisions.
FastAPI API	Expose the agent as the same application rather than creating a separate stack.

17.2 Proposed Agentic RAG phase after this project



- [] First agentic experiment: tool-calling agent with one retrieval tool.
- [] Second: query rewrite only when retrieval confidence is low.
- [] Third: multi-step retrieval with evidence checking.
- [] Measure additional latency and token cost against the non-agent baseline.
- [] Keep a deterministic non-agent fallback for simple queries.

18. Break-friendly operating model and tracking sheet

18.1 Weekly tracker

Week	Start date	Target completion	Actual completion	Status	Main learning	Next action
------	------------	-------------------	-------------------	--------	---------------	-------------

0				Not started		
1				Not started		
2				Not started		
3				Not started		
4				Not started		
5				Not started		
6				Not started		
7				Not started		
8				Not started		
9				Not started		

18.2 How to pause safely

- [] Commit working code before stopping for office work.
- [] Write a 5-line HANDOFF.md note: what works, what is broken, what was last tested, current command, next task.
- [] Record experiment configuration before stopping.
- [] Do not leave a cloud service or API key in an undocumented state.
- [] When restarting after a break, run the last smoke test before touching new code.
- [] Re-read only the current week section rather than restarting the entire course/project.

18.3 Suggested session sizes

Available time	What to do
20-30 min	Read one concept, make one small code change, run one test, commit.
45-60 min	Complete one atomic task from the checklist and update notes.
90-120 min	Implement a component plus unit tests or run a small experiment.
2-4 hours	Run a full experiment cycle, refactor a module, or complete a deployment milestone.

19. Troubleshooting playbook

Problem	First checks	Likely next action
Qdrant connection fails	URL, API key, cluster state, network	Run a standalone connectivity script before debugging the RAG pipeline.
No relevant retrieval	Inspect raw chunks, embedding model, chunk size, query text	Compare dense vs BM25; inspect top 20 results manually.
Hybrid worse than dense	Fusion implementation, score/rank normalization, candidate overlap	Use RRF first; inspect rank lists before adding weights.
Reranker is slow	Candidate N, model size, provider rate limit	Reduce candidate N; compare local vs Cohere; record P95 latency.
LLM hallucinates	Context quality, prompt, provider/model	Inspect retrieved evidence first; then improve grounding/abstention.
Render deployment crashes	Start command, PORT binding, missing env vars, dependency install	Read first error in deploy log; reproduce with the same start command locally.
Render app is slow after idle	Free service sleep/cold start	Document it; keep first public request lightweight.
GitHub Actions fails but local works	Python version, OS dependency, hidden local state	Reproduce in a clean environment and reduce hidden assumptions.
Docker build is confusing	Read Dockerfile top-to-bottom; inspect build logs	Build a tiny hello-world container before changing the project image.

Debugging rule: Always reduce the problem to the smallest layer that can fail: dataset -> chunker -> embedding -> vector search -> BM25 -> fusion -> reranker -> prompt -> LLM -> API -> deployment. Do not debug the entire stack simultaneously.

20. Final definition of done

- [] The app is publicly reachable.
- [] The app returns grounded answers with citations.
- [] The vector store is externalized to Qdrant Cloud for deployment.
- [] The app can switch between local and hosted model providers through configuration.
- [] Dense, BM25, hybrid and reranked retrieval are implemented as separate components.
- [] A benchmark report demonstrates actual retrieval results.
- [] Latency and basic operational metrics are recorded.
- [] Unit/integration tests pass.
- [] GitHub Actions is green.
- [] Docker setup is documented and tested.
- [] Secrets are not committed.
- [] README contains quickstart, architecture, evaluation and demo link.
- [] Dataset attribution/licensing information is present.
- [] At least one technical blog post is published; ideally three linked articles form a series.
- [] LinkedIn post links to both GitHub and the live demo.
- [] Agentic RAG extension is documented as the next project phase.

21. Current reference links

Web sources below were checked on 27 September 2026. Platform limits and pricing can change; use the official source again when executing a deployment step.

- [1] Qdrant documentation - <https://qdrant.tech/documentation/>
- [2] Qdrant local mode / framework documentation - <https://qdrant.tech/documentation/frameworks/langchain/>
- [3] Qdrant Cloud free cluster limits - <https://qdrant.tech/documentation/cloud/create-cluster/>
- [4] Qdrant pricing - <https://qdrant.tech/pricing/>
- [5] Cohere API keys and rate limits - <https://docs.cohere.com/v2/docs/rate-limits>
- [6] Render web services - <https://render.com/docs/web-services>
- [7] Render free services - <https://render.com/docs/free>
- [8] GitHub Pages limits / static site context - <https://docs.github.com/en/pages/getting-started-with-github-pages/github-pages-limits>
- [9] GitHub Actions continuous deployment - <https://docs.github.com/en/actions/get-started/continuous-deployment>
- [10] Hugging Face BEIR/FiQA dataset card - <https://huggingface.co/datasets/BeIR/fiqa>
- [11] FiQA dataset license discussion - <https://huggingface.co/datasets/BeIR/fiqa/discussions/3>
- [12] BEIR datasets list - <https://github.com/beir-cellar/beir/wiki/Datasets-available>
- [13] Reference repository used as project architecture inspiration - <https://github.com/Ashok007-cmd/production-grade-rag>

Project principle to keep visible while building

Build it yourself. Measure it yourself. Deploy it yourself. Explain why you made each choice.